

5.6 Eliminating Tail Recursion

The main benefit of a recursive approach to algorithm design is that it allows us to succinctly take advantage of a repetitive structure present in many problems. By making our algorithm description exploit the repetitive structure in a recursive way, we can often avoid complex case analyses and nested loops. This approach can lead to more readable algorithm descriptions, while still being quite efficient.

However, the usefulness of recursion comes at a modest cost. In particular, the Java Virtual Machine must maintain frames that keep track of the state of each nested call. When computer memory is at a premium, it can be beneficial to derive nonrecursive implementations of recursive algorithms.

In general, we can use the stack data structure, which we will introduce in Section 6.1, to convert a recursive algorithm into a nonrecursive algorithm by managing the nesting of the recursive structure ourselves, rather than relying on the interpreter to do so. Although this only shifts the memory usage from the interpreter to our stack, we may be able to further reduce the memory usage by storing the minimal information necessary.

Even better, some forms of recursion can be eliminated without any use of auxiliary memory. One such form is known as *tail recursion*. A recursion is a tail recursion if any recursive call that is made from one context is the very last operation in that context, with the return value of the recursive call (if any) immediately returned by the enclosing recursion. By necessity, a tail recursion must be a linear recursion (since there is no way to make a second recursive call if you must immediately return the result of the first).

Of the recursive methods demonstrated in this chapter, the `binarySearch` method of Code Fragment 5.3 and the `reverseArray` method of Code Fragment 5.7 are examples of tail recursion. Several others of our linear recursions are almost like tail recursion, but not technically so. For example, our `factorial` method of Code Fragment 5.1 is *not* a tail recursion. It concludes with the command:

```
return n * factorial(n-1);
```

This is not a tail recursion because an additional multiplication is performed after the recursive call is completed, and the result returned is not the same. For similar reasons, the `linearSum` method of Code Fragment 5.6, both power methods from Code Fragments 5.8 and 5.9, and the `fibonacciGood` method of Code Fragment 5.13 fail to be tail recursions.

Tail recursions are special, as they can be automatically reimplemented nonrecursively by enclosing the body in a loop for repetition, and replacing a recursive call with new parameters by a reassignment of the existing parameters to those values. In fact, many programming language implementations may convert tail recursions in this way as an optimization.

```

1  /** Returns true if the target value is found in the data array. */
2  public static boolean binarySearchIterative(int[] data, int target) {
3      int low = 0;
4      int high = data.length - 1;
5      while (low <= high) {
6          int mid = (low + high) / 2;
7          if (target == data[mid])                // found a match
8              return true;
9          else if (target < data[mid])
10             high = mid - 1;                    // only consider values left of mid
11          else
12             low = mid + 1;                      // only consider values right of mid
13      }
14      return false;                            // loop ended without success
15  }

```

Code Fragment 5.15: A nonrecursive implementation of binary search.

As a tangible example, our `binarySearch` method can be reimplemented as shown in Code Fragment 5.15. We initialize variables `low` and `high` to represent the full extent of the array just prior to our `while` loop. Then, during each pass of the loop, we either find the target, or we narrow the range of the candidate subarray. Where we made the recursive call `binarySearch(data, target, low, mid - 1)` in the original version, we simply replace `high = mid - 1` in our new version and then continue to the next iteration of the loop. Our original base case condition of `low > high` has simply been replaced by the opposite loop condition, **while** `low <= high`. In our new implementation, we return `false` to designate a failed search if the `while` loop ends without having ever returned `true` from within.

Most other linear recursions can be expressed quite efficiently with iteration, even if they were not formally tail recursions. For example, there are trivial nonrecursive implementations for computing factorials, computing Fibonacci numbers, summing elements of an array, or reversing the contents of an array. For example, Code Fragment 5.16 provides a nonrecursive method to reverse the contents of an array (as compared to the earlier recursive method from Code Fragment 5.7).

```

1  /** Reverses the contents of the given array. */
2  public static void reversesIterative(int[] data) {
3      int low = 0, high = data.length - 1;
4      while (low < high) {                        // swap data[low] and data[high]
5          int temp = data[low];
6          data[low++] = data[high];              // post-increment of low
7          data[high--] = temp;                  // post-decrement of high
8      }
9  }

```

Code Fragment 5.16: Reversing the elements of a sequence using iteration.